ACADEMIC PROJECT DOCUMENTATION REPORT

AI-Based Personal Budget Planning & Financial Goal System

A Multi-Tenant Financial Analytics & Machine Learning Web Platform

Degree Program:	Master of Computer Applications (MCA)
Course Focus:	Final Semester Major Capstone Project
Author / Candidate:	Gayatri Chokhani (GitHub: **@gayatripch20**)
Repository:	https://github.com/gayatripch20/ai-budget-planner
Technology Stack:	Python 3.14, Flask 3.1, MySQL 8.0, Scikit-Learn, Pandas, Bootstrap 5, Chart.js, ReportLab
Verification Matrix:	49 / 49 Pytest Test Cases Passing (100% Pass Rate)
Architecture:	Three-Tier MVC / Service-Oriented Multi-Tenant Architecture
Academic Year:	2026

EXECUTIVE SUMMARY & VIVA HIGHLIGHTS

This document constitutes the official academic report and technical reference for the **AI-Based Personal Budget Planning & Financial Goal System**. Built ground-up without third-party template clones, it features strict decimal accounting precision, multi-tenant data isolation (IDOR-safe), automated 80%/100% budget threshold alerts, mathematical milestone goal amortization, and an academic Scikit-learn multi-tier forecasting engine evaluated via Mean Absolute Error (MAE).

Course: Master of Computer Applications (MCA)

Technology Stack: Python 3, Flask Micro-Framework, MySQL 8.0, SQLAlchemy ORM, Scikit-Learn, Pandas, NumPy, Bootstrap 5, Chart.js, ReportLab

---

1. Project Abstract

Modern personal finance management is frequently hindered by fragmented manual record-keeping, lack of disciplined category spending caps, and inability to forecast upcoming financial obligations. This academic project presents the design and implementation of the **"AI-Based Personal Budget Planning & Financial Goal System"**, an enterprise-grade, secure, multi-tenant web application developed in Python (Flask) and MySQL.

The application allows registered users to track income and expenditure transactions across categorized ledgers with exact fixed-point **`DECIMAL(12, 2)`** precision. Users define monthly category budgets that monitor consumption in real time and automatically trigger threshold alerts at 80% (warning) and 100% (exceeded). A milestone savings goal management engine calculates required monthly amortized contributions based on target deadlines ($S_{monthly} = \frac{\text{Target} - \text{Current}}{\text{Months Remaining}}$).

An integrated Machine Learning module built with **Scikit-Learn** and **Pandas** implements a multi-tiered forecasting strategy: naive historical moving averages for cold-start users ($N < 3$ months) and Regularized Ridge / Linear Trend Regression with lag and cyclical month features for users with sufficient historical depth, evaluated strictly via Mean Absolute Error (MAE). A deterministic advisory engine analyzes financial health against the **50/30/20 rule** and detects runaway category velocities. Financial statements are exportable as RFC 4180 CSV files and publication-quality PDF audit reports generated in memory via ReportLab. The application is verified by an automated test suite of 49 test cases with zero security defects.

---

2. Introduction

In an increasingly digital economy characterized by frictionless payments (UPI, credit cards, net banking), individual cash outflows often occur impulsively. Without centralized visibility, individuals face month-end liquidity shortages, chronic budget overruns, and missed long-term milestones.

This application provides a holistic financial dashboard integrating transaction accounting, automated budget enforcement, predictive expenditure projection, and milestone savings management into an intuitive, accessible web interface.

---

3. Problem Statement

1. **Lack of Centralized Tracking:** Expenses spread across multiple payment modes (UPI, cards, cash) remain unaggregated.
2. **Delayed Overspending Feedback:** Traditional budgets are reviewed retrospectively after month-end when corrective action is impossible.
3. **Imprecise Goal Amortization:** Savers struggle to determine how much money must be set aside monthly to meet upcoming financial deadlines.
4. **Data Scarcity in Financial AI:** Applying complex deep learning models to sparse personal financial records yields untrustworthy, overfitted forecasts.

---

4. Objectives

- Develop an isolated, multi-tenant web application using Python 3 and Flask.
- Implement a 3NF normalized MySQL database with InnoDB engine to guarantee ACID transaction integrity.
- Deliver automated real-time budget tracking with proactive threshold alerts (80% and 100%).
- Formulate an automated goal amortization calculation engine.
- Engineer an academically rigorous, multi-tiered Scikit-Learn forecasting model evaluated via MAE.
- Implement a deterministic smart recommendation system based on the 50/30/20 budgeting rule.
- Provide in-memory export streaming for CSV and PDF audit reports without persisting server-side files.
- Attain 100% test pass rate across unit, functional, security, and edge-case suites.

---

5. Scope of the Project

- **Target Audience:** Students, young working professionals, and households seeking disciplined financial tracking.
- **In-Scope:** Authentication, multi-tenant isolation, transaction accounting, category budgets, milestone savings goals, Chart.js visual analytics, Scikit-learn forecasting, rule-based recommendations, and CSV/PDF export.
- **Academic Boundary / Out-of-Scope:** Commercial Open Banking APIs (e.g. Plaid, Yodlee) requiring banking licenses. Real-world simulation is supported via standard imports and manual accounting.

---

6. Existing System vs. Proposed System

Feature / Metric	Existing System (Spreadsheets / Paper / Manual Apps)	Proposed System (AI Budget Planner)
Data Organization	Disjointed spreadsheets or offline diaries prone to loss.	Centralized 3NF normalized MySQL database with ACID compliance.
Budget Enforcement	Passive and retrospective; calculated days after month-end.	Active real-time progress bars with automated 80%/100% alert triggers.
Forecasting Ability	None, or basic intuition without mathematical backing.	Multi-tier Scikit-learn regression models with baseline MAE evaluation.
Goal Planning	Manual arithmetic prone to deadline miscalculations.	Automated monthly savings amortization formula.
Advisory Engine	Generic, static one-size-fits-all tips.	Real-time rules assessing 50/30/20 compliance and runaway category velocities.
Audit & Export	Manual screenshotting or manual table copying.	Instant RFC 4180 CSV streaming & ReportLab vector PDF generation.

---

7. Functional and Non-Functional Requirements

Functional Requirements

- **FR1 (Authentication):** User registration, password hashing (PBKDF2/SHA256), session state management, and logout.
- **FR2 (Transaction Ledger):** Add, edit, delete, and paginate income and expense transactions with date, category, and payment mode.

- **FR3 (Budget Management):** Define category spending caps per month/year with composite unique constraints.
- **FR4 (Goal Tracking):** Create savings targets with deadline countdowns and deposit contribution logs.
- **FR5 (Analytics):** Render Month-over-Month (MoM) variance, 6-month comparative trends, and category distribution donut charts.
- **FR6 (AI Forecasting):** Compute next month's projected expense using Scikit-Learn regression or cold-start baseline.
- **FR7 (Export):** Stream in-memory CSV and PDF audit statements.

Non-Functional Requirements

- **NFR1 (Security):** CSRF tokens on all state-altering forms; parameterized queries via SQLAlchemy; Insecure Direct Object Reference (IDOR) prevention.
- **NFR2 (Accuracy):** High-precision `DECIMAL(12, 2)` monetary arithmetic eliminating binary floating-point roundoff errors.
- **NFR3 (Performance):** Compound indexes on `(user_id, transaction_date)` enabling sub-50ms database lookup times.
- **NFR4 (Responsiveness):** Mobile-first responsive UI built on Bootstrap 5 and accessible via standard web browsers.

---

8. Software and Hardware Requirements

Software Environment

- **Operating System:** Windows 10 / 11, Linux, or macOS
- **Backend Language:** Python 3.10+ (Tested on Python 3.14.5)
- **Web Framework:** Flask 3.1.3
- **Database Engine:** MySQL Server 8.0 (InnoDB storage engine)
- **ORM / Migrations:** Flask-SQLAlchemy 3.1.1, Flask-Migrate 4.1.0 (Alembic 1.20.1)
- **Machine Learning & Analytics:** Scikit-Learn 1.9.1, Pandas 3.1.0, NumPy 2.5.4
- **Document Generation:** ReportLab 5.1.0
- **Testing Framework:** Pytest 9.1.1

Hardware Environment

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher
- **Memory (RAM):** Minimum 4 GB (8 GB recommended)
- **Disk Storage:** Minimum 500 MB free space for code, virtual environment, and database

---

9. Feasibility Study

- **Technical Feasibility:** Python and Flask are mature, industry-standard web technologies. Scikit-learn and Pandas integrate natively into Flask routes.
- **Operational Feasibility:** The web UI requires no special software installation for end users—only a standard web browser.
- **Economic Feasibility:** The tech stack is 100% open-source (MIT, BSD, and Apache licenses). No proprietary commercial software licenses are required.

---

10. Software Development Life Cycle (SDLC) Methodology

The project adopted the **Iterative and Incremental Agile Methodology**. Development was structured across 12 distinct sequential phases. Each iteration added a functional, tested module accompanied by automated unit tests and progressive Git commits:

1. Requirements & Architecture $\rightarrow$ 2. Environment Setup $\rightarrow$ 3. Database & Models $\rightarrow$ 4. Auth & Security $\rightarrow$ 5. Transactions $\rightarrow$ 6. Budgets & Goals $\rightarrow$ 7. Dashboard & Analytics $\rightarrow$ 8. AI/ML Forecasting $\rightarrow$ 9. Reports & Exports $\rightarrow$ 10. Testing & Security Audit $\rightarrow$ 11. Academic Documentation $\rightarrow$ 12. Final Delivery.

---

11. System Architecture Diagram

```
flowchart TD
    Client["Client Web Browser (Bootstrap 5, Chart.js)"]
    WebRouter["Flask WSGI Router & Blueprints"]
    AuthLayer["Flask-Login & CSRF Protection Layer"]
    ServiceLayer["Business Services Layer\n- Amortization Engine\n- 50/30/20 Recommender\n- Report Generator"]
    MLEngine["ML Analytics Engine\n- Pandas Preprocessing\n- Scikit-Learn Regressor"]
    ORM["SQLAlchemy ORM (PyMySQL Driver)"]
    DB[("MySQL 8.0 Database\n(InnoDB, 3NF Schema)")]

    Client -->|HTTP / Forms| WebRouter
    WebRouter --> AuthLayer
    AuthLayer --> ServiceLayer
    ServiceLayer --> MLEngine
    ServiceLayer --> ORM
    MLEngine --> ORM
    ORM -->|Parameterized SQL| DB
    DB -->|Result Sets| ORM
    ORM --> ServiceLayer
    ServiceLayer --> WebRouter
    WebRouter -->|Jinja2 Templates & JSON Payloads| Client
```

---

12. Use-Case Diagram

```
flowchart LR
    User((Authenticated User))

    UC1["Register / Log In"]
    UC2["Log Income & Expense Transactions"]
    UC3["Filter & Search Ledger"]
    UC4["Define Category Budgets"]
    UC5["Create & Fund Savings Goals"]
    UC6["View Interactive Analytics & Trends"]
    UC7["Generate AI Expense Forecast"]
    UC8["Receive Smart Recommendations"]
    UC9["Export CSV / Download PDF Report"]
    UC10["Manage Profile & Currency"]

    User --> UC1
    User --> UC2
    User --> UC3
    User --> UC4
    User --> UC5
    User --> UC6
    User --> UC7
    User --> UC8
    User --> UC9
    User --> UC10
```

13. Data Flow Diagrams (DFD)

Context Level (Level 0 DFD)

```
flowchart LR
    UserEntity["User / Account Holder"]
    SystemProcess["(0.0)\nAI-Based Personal Budget\n& Financial Goal System"]
    MySQLStore[("MySQL\nFinancial Database")]

    UserEntity -->|Credentials, Transactions, Budgets, Goals| SystemProcess
    SystemProcess -->|Dashboard KPIs, Charts, Forecasts, PDF/CSV Reports| UserEntity
    SystemProcess <-->|Parameterized Queries & Result Records| MySQLStore
```

Level 1 DFD

```
flowchart TD
    User["User"]

    P1["(1.0) Auth & Session Controller"]
    P2["(2.0) Transaction Ledger Manager"]
    P3["(3.0) Budget Enforcement & Alert Dispatcher"]
    P4["(4.0) Goal Amortization Engine"]
    P5["(5.0) ML Forecasting & 50/30/20 Recommender"]
    P6["(6.0) Report & Export Streamer"]

    DB_Users[("users")]
    DB_Tx[("transactions")]
    DB_Budgets[("budgets")]
    DB_Goals[("financial_goals")]
    DB_Pred[("expense_predictions")]
    DB_Notif[("notifications")]

    User -->|Login/Register| P1
    P1 <--> DB_Users

    User -->|Post Transaction| P2
    P2 <--> DB_Tx
    P2 -->|Notify Inflow/Outflow| P3

    User -->|Set Budget Limit| P3
```

```
    P3 <--> DB_Budgets
    P3 -->|Breach Alert| DB_Notif

    User -->|Create Goal / Deposit| P4
    P4 <--> DB_Goals

    P2 -->|Historical Expenses| P5
    P5 <--> DB_Pred
    P5 -->|Advice| DB_Notif

    User -->|Request Statement| P6
    P6 <--> DB_Tx
    P6 -->|Stream PDF/CSV| User
```

14. Entity-Relationship (ER) Schema

```
erDiagram
    USERS ||--o{ TRANSACTIONS : "records"
    USERS ||--o{ BUDGETS : "defines"
    USERS ||--o{ FINANCIAL_GOALS : "tracks"
    USERS ||--o{ NOTIFICATIONS : "receives"
    USERS ||--o{ EXPENSE_PREDICTIONS : "generates"
    CATEGORIES ||--o{ TRANSACTIONS : "classifies"
    CATEGORIES ||--o{ BUDGETS : "constrains"
    FINANCIAL_GOALS ||--o{ SAVINGS_CONTRIBUTIONS : "accumulates"

    USERS {
        int id PK
        varchar(100) full_name
        varchar(120) email UK
        varchar(255) password_hash
        varchar(10) currency
        datetime created_at
        datetime updated_at
    }

    CATEGORIES {
        int id PK
        int user_id FK "Nullable for system categories"
        varchar(50) name
        varchar(20) type
```

```
        varchar(50) icon
        varchar(20) color
        boolean is_default
    }

    TRANSACTIONS {
        int id PK
        int user_id FK
        int category_id FK
        varchar(20) transaction_type
        decimal(12_2) amount
        date transaction_date
        varchar(50) payment_method
        varchar(255) description
    }

    BUDGETS {
        int id PK
        int user_id FK
        int category_id FK
        int month
        int year
        decimal(12_2) amount_limit
    }
```

```
    FINANCIAL_GOALS {
        int id PK
        int user_id FK
        varchar(120) title
        decimal(12_2) target_amount
        decimal(12_2) current_amount
        date target_date
        varchar(20) status
    }

    SAVINGS_CONTRIBUTIONS {
        int id PK
        int goal_id FK
        int user_id FK
        decimal(12_2) amount
        date contribution_date
        varchar(255) notes
    }

    NOTIFICATIONS {
        int id PK
        int user_id FK
        varchar(150) title
        text message
        varchar(20) notification_type
```

```
        boolean is_read
    }

    EXPENSE_PREDICTIONS {
        int id PK
        int user_id FK
        int forecast_month
        int forecast_year
        decimal(12_2) predicted_amount
        varchar(50) model_used
        decimal(8_2) mae_score
    }
```

15. Database Data Dictionary

Table 1: `users`

Column Name	Data Type	Constraints	Description
`id`	INT	Primary Key, Auto Increment	Unique account identifier
`full_name`	VARCHAR(100)	NOT NULL	User's legal full name
`email`	VARCHAR(120)	NOT NULL, UNIQUE, INDEX	Login email address
`password_hash`	VARCHAR(255)	NOT NULL	PBKDF2/SHA256 password hash
`currency`	VARCHAR(10)	NOT NULL, Default '■'	Preferred currency symbol
`created_at`	DATETIME	NOT NULL	Account registration timestamp
`updated_at`	DATETIME	NOT NULL	Account update timestamp

Table 2: `categories`

Column Name	Data Type	Constraints	Description

id	INT	Primary Key, Auto Increment	Unique category identifier
user_id	INT	Foreign Key (**users.id**), NULLABLE	NULL for system defaults; user ID for custom
name	VARCHAR(50)	NOT NULL	Category title (e.g. Salary, Groceries)
type	VARCHAR(20)	NOT NULL	'income' or 'expense'
icon	VARCHAR(50)	NOT NULL, Default 'bi-tag'	Bootstrap icon CSS class
color	VARCHAR(20)	NOT NULL, Default '#4e73df'	HEX color code for Chart.js
is_default	BOOLEAN	NOT NULL, Default FALSE	System default category flag

Table 3: `transactions`

Column Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique transaction record ID
user_id	INT	Foreign Key (**users.id**), NOT NULL	Owner user ID
category_id	INT	Foreign Key (**categories.id**), NOT NULL	Associated category classification
transaction_type	VARCHAR(20)	NOT NULL	'income' or 'expense'
amount	DECIMAL(12, 2)	NOT NULL	High-precision monetary value
transaction_date	DATE	NOT NULL, INDEX	Transaction calendar date
payment_method	VARCHAR(50)	NOT NULL	Cash, UPI, Debit Card, Credit Card, etc.
description	VARCHAR(255)	NULLABLE	Optional memo or note
created_at	DATETIME	NOT NULL	Timestamp when logged

Table 4: `budgets`

Column Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique budget cap ID
user_id	INT	Foreign Key (**users.id**), NOT NULL	Owner user ID
category_id	INT	Foreign Key (**categories.id**), NOT NULL	Expense category being limited
month	INT	NOT NULL (1 to 12)	Calendar month
year	INT	NOT NULL	Calendar year (e.g. 2026)
amount_limit	DECIMAL(12, 2)	NOT NULL	Allocated maximum spending cap

Table 5: `financial_goals`

Column Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique financial goal ID
user_id	INT	Foreign Key (**users.id**), NOT NULL	Owner user ID

title	VARCHAR(120)	NOT NULL	Goal title (e.g. New Laptop)
target_amount	DECIMAL(12, 2)	NOT NULL	Total savings target
current_amount	DECIMAL(12, 2)	NOT NULL, Default 0.00	Current accumulated savings
target_date	DATE	NOT NULL	Target deadline date
status	VARCHAR(20)	NOT NULL, Default 'in_progress'	'in_progress', 'completed', 'overdue'
notes	TEXT	NULLABLE	Motivation or savings plan description

Table 6: savings_contributions

Column Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique contribution ID
goal_id	INT	Foreign Key (**financial_goals.id**), NOT NULL	Parent goal
user_id	INT	Foreign Key (**users.id**), NOT NULL	Depositor user ID
amount	DECIMAL(12, 2)	NOT NULL	Deposited contribution amount
contribution_date	DATE	NOT NULL	Date of deposit
notes	VARCHAR(255)	NULLABLE	Note (e.g. Bonus deposit)

Table 7: notifications

Column Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique alert ID
user_id	INT	Foreign Key (**users.id**), NOT NULL	Recipient user ID
title	VARCHAR(150)	NOT NULL	Alert title
message	TEXT	NOT NULL	Alert text / recommendation details
notification_type	VARCHAR(20)	NOT NULL, Default 'info'	'info', 'warning', 'danger', 'success'
is_read	BOOLEAN	NOT NULL, Default FALSE	Read / unread status indicator

Table 8: expense_predictions

Column Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique prediction record ID
user_id	INT	Foreign Key (**users.id**), NOT NULL	Target user ID
forecast_month	INT	NOT NULL	Upcoming month predicted
forecast_year	INT	NOT NULL	Upcoming year predicted
predicted_amount	DECIMAL(12, 2)	NOT NULL	Projected expense total
model_used	VARCHAR(50)	NOT NULL	Algorithm used ('LinearTrendRegression', etc.)

mae_score	DECIMAL(8, 2)	NULLABLE	Mean Absolute Error validation score

16. Key Algorithms & Mathematical Formulations

Algorithm 1: Financial Goal Amortization Calculation

```
Input: target_amount, current_amount, target_date, today
Output: remaining_amount, percentage_completed, required_monthly_savings

1. remaining_amount = max(0.00, target_amount - current_amount)
2. percentage_completed = min(100.0, (current_amount / target_amount) * 100.0)
3. diff_years = target_date.year - today.year
4. diff_months = target_date.month - today.month
5. total_months = (diff_years * 12) + diff_months
6. months_remaining = max(1, total_months)
7. required_monthly_savings = round(remaining_amount / months_remaining, 2)
8. Return {remaining_amount, percentage_completed, required_monthly_savings}
```

Algorithm 2: Multi-Tiered AI Expense Forecasting

```
Input: user_id, historical_transactions
Output: predicted_amount, model_used, mae_score, confidence

1. df = group_by_month(historical_transactions)
2. N = len(df)
3. If N == 0:
       Return {predicted_amount: 0.00, model: "NoData", confidence: "none"}
4. Else if N < 3:  // Tier 1: Cold Start
       predicted_amount = mean(df['total_expense'])
       Return {predicted_amount, model: "MovingAverageBaseline", confidence: "low", mae: None}
5. Else if 3 <= N < 6: // Tier 2: Linear Trend
       model = LinearRegression().fit(X=period_index, y=total_expense)
       predicted_amount = model.predict(N + 1)
       mae = compute_mae(y, model.predict(X))
       Return {predicted_amount, model: "LinearTrendRegression", confidence: "moderate", mae}
6. Else (N >= 6): // Tier 3: Time-Series Regularized Ridge
       X = [period_index, lag_1, sin(2*pi*m/12), cos(2*pi*m/12)]
       X_train, y_train = X[:-1], y[:-1]
       X_test, y_test = X[-1:], y[-1:]
       model = Ridge(alpha=1.0).fit(X_train, y_train)
       mae = compute_mae(y_test, model.predict(X_test))
       model.fit(X, y)
       predicted_amount = model.predict(X_next)
       Return {predicted_amount, model: "RidgeTimeSeriesRegression", confidence: "high", mae}
```

17. Test Plan & Actual Results Matrix

Test Suite File	Test Cases Executed	Coverage Area	Status
test_environment.py	3	Python 3.10+ check, Core library imports, Module structure	**PASSED**
test_models.py	6	Password hashing, Category relations, Budget uniqueness, Goal amortization math	**PASSED**

`test_auth.py`	7	Registration, duplicate email rejection, login/logout, route protection	**PASSED**
`test_transactions.py`	6	Income/expense CRUD, negative amount rejection, IDOR cross-user check	**PASSED**
`test_budgets_and_goals.py`	5	80%/100% threshold alert notifications, goal contributions, auto-completion	**PASSED**
`test_dashboard_and_analytics.py`	6	Savings rate formula, category donut breakdown, 6-month trend line, alert dismiss	**PASSED**
`test_ml_and_recommendations.py`	5	Cold-start handling, Scikit-learn Linear/Ridge regression, 50/30/20 rule diagnostics	**PASSED**
`test_reports_and_profile.py`	6	RFC 4180 CSV export, ReportLab `%PDF` magic header, profile & currency updates	**PASSED**
`test_qa_and_security.py`	5	Unauthorized access across all modules, 404 handler, empty dataset resilience	**PASSED**
TOTAL	**49**	**Complete System End-to-End Verification**	**100% PASSED**

---

18. Limitations

1. **Manual / CSV Data Entry:** Open Banking API integrations are excluded due to real-world financial licensing constraints.
2. **Short-Term Time Series Sensitivity:** Users with fewer than 3 months of history rely on statistical moving average baselines until sufficient records accumulate.
3. **Currency Conversion:** Multi-currency support applies symbol changes globally but does not execute real-time Forex exchange conversions.

---

19. Future Enhancements

1. **Receipt OCR Scanning:** Integrate optical character recognition (e.g. Tesseract OCR) to extract amount and vendor directly from bill photos.
2. **SMS / Email Transaction Parsing:** Implement automated local inbox regex parsers for bank transaction confirmation alerts.
3. **Advanced Time-Series Models:** Integrate ARIMA / Prophet / LSTM neural networks when users accumulate more than 24 months of continuous historical data.

---

20. Conclusion

The **AI-Based Personal Budget Planning & Financial Goal System** fulfills all requirements of a rigorous, secure, production-grade MCA final project. Built strictly from scratch without third-party template clones, it demonstrates comprehensive mastery of full-stack Python development, normalized MySQL database architecture, scikit-learn machine learning, data visualization, and defensive cybersecurity principles.